KOENIG

step forward



Null Safety

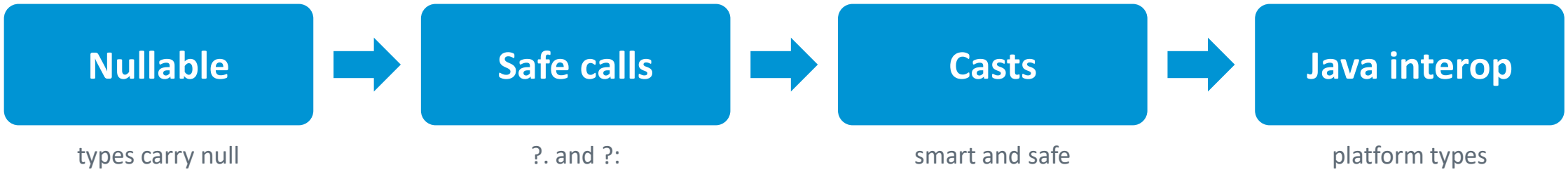
Kotlin for Java Developers

What You Will Learn

1. Nullable and non-nullable types
2. ?. , ?: and the not-null assertion
3. Safe casts and smart casts
4. Scope functions: let, run, also, apply, with
5. Platform types, @Nullable and @NotNull

Demo Scenario: ShopKart Without NullPointerException

Not every ShopKart shopper card is complete: Dev has no address, Chitra no phone, and old Java code returns null without a warning. We make the Kotlin side null-safe, one risk at a time.



We will cut

- **NPE crashes**
- **Nested ifs**
- **Unsafe !!**

The Billion-Dollar Mistake

- **Null** was added to languages in 1965; its inventor called it a billion-dollar mistake.
- **In Java** any reference can be null and the compiler says nothing.
- **The result:** NullPointerException at run time, often in production.
- **Kotlin's answer:** make null part of the type system.

```
String city = shopper.getAddress().getCity();
// compiles fine
// NullPointerException when the
// shopper has no address

String phone = shopper.getPhone();
int n = phone.length();   // may crash too
```

Java finds null problems in production. Kotlin finds them in the editor

Nullable vs Non-Null Types

- **String** can never be null. The compiler enforces it.
- **String?** may hold null. The ? is part of the type.
- **Same for** Int and Int?, Shopper and Shopper?, List<String> and List<String>?.
- **Rule:** a nullable value cannot be used until you handle the null.

```
var name: String = "Asha"
// name = null              // does not compile

var nick: String? = "Ash"
nick = null                 // fine

// nick.length              // does not compile
val n: Int? = nick?.length   // Int? : may be null
```

The type tells you whether null is possible

Java vs Kotlin: Null Handling

	Java	Kotlin
Declare	Any reference may be null	String vs String? in the type
Check	if (x != null) { ... }	x?.length and smart casts
Default	x != null ? x : "n/a"	x ?: "n/a"
Failure	NPE at run time	error in the editor

Same ideas, but Kotlin checks them for you

Demo: Nullable and Non-Null Types

What we build

- **String and String?:** two different types.
- **Compiler errors:** null, .length.

How the concept helps

- **Editor:** finds the crash early.
- **Type:** says whether null is possible.
- **Java:** needed run time to learn it.

Four Ways to Handle a Nullable

Check

if (x != null) and the compiler smart-casts x inside.

Safe call ?.

Call only if not null; otherwise the result is null.

Elvis ?:

Supply a default, or leave the function early.

let { }

Run a block only when the value is present.

```
val c: Shopper? = find(101)
if (c != null) println(c.name)       // check
println(c?.name)                     // safe call
println(c?.name ?: "nobody")         // Elvis
c?.let { println("found ${it.name}") }   // let
```

Each replaces a block of if-null code and none of them crashes

The Safe Call Operator ?.

- **a?.b** means: if a is not null then a.b, otherwise null.
- **The result** type is always nullable.
- **Works for** properties and method calls.
- **Never throws** a NullPointerException.

```
val shopper: Shopper? = find(101)

val name: String? = shopper?.name
val city: String? = shopper?.address?.city
val digits: Int? = shopper?.phone?.length

// shopper?.name is the same as:
//   if (shopper != null) shopper.name else null
```

A question mark before the dot stops the crash

Chaining Safe Calls

Java

```
String code = null;
Shopper s = find(id);
if (s != null) {
    DeliveryAddress a = s.getAddress();
    if (a != null) {
        code = a.getPostcode();
    }
}
```

Kotlin

```
val code: String? =
    find(id)?.address?.postcode
```

- One null anywhere in the chain makes the whole chain null.
- The rest of the chain is skipped (short-circuit).
- Eight lines of Java become one statement of Kotlin.

Nested ifs collapse into one readable chain

Demo: Safe Calls on Shopper Cards

What we build

- **shopper?.name:** null when no shopper.
- **Chain:** address?.city, phone?.length.
- **postcodeOf():** one line, no nesting.

How the concept helps

- **No crash:** null travels forward.
- **Short:** eight lines become one.
- **Honest:** the result is nullable.

The Elvis Operator ?:

- **a ?: b** means: a if it is not null, otherwise b.
- **Named** after Elvis: turn your head sideways.
- **Pairs with ?.** safe call gives null, Elvis replaces it.
- **Right side** is only evaluated when needed.

```
val city = shopper?.address?.city ?: "no city"

val n = nickname?.length ?: 0

// Java equivalent
//   String city = x != null ? x : "no city";
```

?. makes it nullable, ?: makes it non-null again

Elvis with return and throw

```
fun label(id: Int): String {
    val shopper = find(id) ?: return "UNKNOWN"
    val address = shopper.address ?: return "no address"
    return "${shopper.name}, ${address.city}"
}

fun mustFind(id: Int): Shopper =
    find(id) ?: throw IllegalArgumentException("shopper $id not found")

fun mustFind2(id: Int): Shopper =
    requireNotNull(find(id)) { "shopper $id is required" }
```

- **return** and throw are expressions of type Nothing, so they fit on the right.
- **Early exit:** after the line, shopper and address are non-null (smart cast).
- **requireNotNull** gives a clear message.

Guard clauses replace deep nesting

Demo: Elvis, Early Return and requireNotNull

What we build

- **?: "no city":** a default value.
- **label():** return on the right side.
- **mustFind():** throw and requireNotNull.

How the concept helps

- **Non-null again:** after ?: the type is String.
- **Guard clauses:** no pyramid of ifs.
- **Clear message:** the failing id is named.

The Not-Null Assertion !!

```
val shopper: Shopper? = find(104)

val city = shopper!!.address!!.city
// NullPointerException (no helpful message)
```

- **x!!** means: I promise x is not null; crash if I am wrong.
- **It turns** compile-time safety back into a run-time crash.
- **Hard to debug:** which of the two was null?
- **Treat it as** a code smell; every !! is a bug waiting for the right input.

Demo goal: replace every !! with a safe form

Replacing the Assertion

x ?: default

a safe fallback value

x ?: return

leave the function early

x?.let { ... }

do work only when present

requireNotNull(x)

fail early with a clear message

Change the type

make the value non-null at its source

Smart cast

check once, then use as non-null

Ask first: should this value be nullable at all?

Demo: The !! Crash and Its Replacements

What we build

- **shopper!!.address!!:** NullPointerException.
- **?: and if:** default and smart cast.
- **requireNotNull:** a clear message.

How the concept helps

- **Which one was null?** !! cannot tell.
- **Alternatives:** all keep compile-time safety.
- **Habit:** no !! in ShopKart code.

Smart Casts

```
fun describe(o: Any) = when (o) {
    is String -> "text of length ${o.length}"
    is Int -> "number ${o + 1}"
    else -> "other"
}
```

- **After a check** the compiler remembers the type.
- **Works with** != null, is, && and ||, early return and when.
- **No cast** needed, unlike Java instanceof plus (String) cast.

Check once, use freely

When Smart Casts Do Not Work

```
class Box(var value: String?)

fun lengthOf(box: Box): Int {
    // if (box.value != null) box.value.length
    //   error: smart cast impossible, value is a
    //   mutable property

    val v = box.value                // copy to a local val
    return if (v != null) v.length else 0
}

fun lengthOf2(box: Box): Int = box.value?.length ?: 0
```

- **Works for:** local val and var (if not captured and changed), val properties in the same module.
- **Fails for:** var properties, custom getters, properties of another module.
- **Fix:** local copy, ?. or let.

The compiler only trusts values that cannot change

Safe Casts: as?

Java

```
Object o = "text";
Shopper s = (Shopper) o;
// ClassCastException at run time

if (o instanceof Shopper s2) {
    ...
}
```

Kotlin

```
val o: Any = "text"
val a = o as? Shopper              // null
// val b = o as Shopper            // ClassCastException

val len = (o as? String)?.length ?: 0
val s: Shopper? = o as? Shopper
```

- **as** throws ClassCastException when the type is wrong
- **as?** returns null instead. Combine with ?. and ?:
- **Prefer is** with smart cast when you also need the else branch

as? turns a crash into a null you can handle

Demo: Smart Casts and as?

What we build

- **show():** String? checked once.
- **is and when:** cast inside the branch.
- **as?:** null instead of an exception.

How the concept helps

- **No casts:** the compiler remembers.
- **var property:** copy it to a local val.
- **as? plus ?:** a crash becomes a value.

Lambdas in Kotlin

- **{ } is a lambda:** a block you pass to a function.
- **it** is the single parameter; or name it, p ->.
- **Last expression** is the result: no return.

```kotlin
val phone: String? = "98765 43210"

phone?.let { println("calling $it") }
// it: the single parameter

val digits = phone?.let { p ->
    p.length
} ?: 0
// or name the parameter: p ->

// Java: p -> ...      Kotlin: { p -> ... }
// the last expression is the result
```

?.let { } reads: if present, then do this

Scope Functions at a Glance

Function	Object is	Returns	Typical use
let	it	lambda result	null checks, transform
run	this	lambda result	compute from the object
with	this	lambda result	group calls on an object
apply	this	the object	configure, then return it
also	it	the object	side effects, logging

Two questions: it or this? Result or object?

let and run

```
// let: it, returns the lambda result
asha.phone?.let { println("calling $it") }

val digits = asha.phone?.let { p ->
    p.length
} ?: 0

// run: this, returns the lambda result
val summary = dev.run {
    "$name lives in ${address?.city ?: "unknown"}"
}
```

- **let** is the null-check tool: ?.let { } runs only for non-null.
- **Name the** parameter when lambdas nest.
- **run** is let with this instead of it.

?.let { } means: if present, then do this

also and apply

```
// also: it, returns the object (side effects)
val loaded = find(103)?.also { println("loaded ${it.name}") }

// apply: this, returns the object (configure)
val lines = mutableListOf<String>().apply {
    add("name=Asha")
    add("id=101")
}

val sb = StringBuilder().apply {
    append("Hello, ")
    append("Asha")
}.toString()
```

- **also** for logging or debugging in the middle of a chain.
- **apply** for object setup: builder style without a builder.
- **Both** return the same object, so they chain.

also observes the object, apply configures it

with and Choosing One

```
// with: this, returns the lambda result
// not an extension: the object is an argument
val text = with(asha) {
    "$name <${email ?: "no email"}>"
}
```

- **with** is for a non-null object you use several times.
- **Do not nest** scope functions deeply: this and it become confusing.

Need a null check?

let

Configure and return?

apply

Log in the middle?

also

Compute from an object?

run or with

Pick for readability, not for cleverness

Demo: let, run, also, apply and with

What we build

- **let:** phone only when present.
- **run:** summary with this.
- **also, apply, with:** log, configure, compute.

How the concept helps

- **Null checks:** ?.let replaces if.
- **Two questions:** it or this, result or object.
- **Readability:** no deep nesting.

Platform Types from Java

```
// Java: no annotations
public String findNickname(int id) { ... }
```

```
val a = legacy.findNickname(104)      // String!
println(a.length)                     // compiles,
                                      // NPE at run time

val b: String? = legacy.findNickname(104)
println(b?.length ?: 0)
```

- **String!** is a platform type: the compiler cannot tell if it is nullable.
- **Kotlin** relaxes its checks so Java interop stays convenient.
- **Danger:** the null check you skip comes back as a NullPointerException.
- **Safe habit:** declare the type yourself and write String?.

String! means: Java did not tell Kotlin the truth

@Nullable and @NotNull

Java with annotations

```
import org.jetbrains.annotations.*;

public class AnnotatedNicknames {
    @Nullable
    public String findNickname(int id) { ... }

    @NotNull
    public String greeting(@NotNull String name) {
        ...
    }
}
```

Kotlin sees

```
val nick = repo.findNickname(104)    // String?
// nick.length                       // error
println(nick?.length ?: 0)

println(repo.greeting("Ben"))        // String
// repo.greeting(null)               // error
```

Annotations make Java honest about null

Demo: Platform Types and Annotations

What we build

- **LegacyNicknames:** String! and a late NPE.
- **@Nullable, @NotNull:** the compiler enforces them.

How the concept helps

- **Honest Java:** annotations carry null.
- **Safe habit:** declare String? yourself.

Key Takeaways

1. **Types carry null**
   String vs String?; the compiler checks it.

2. **Handle, do not assert**
   ?. , ?: and let replace !! and nested ifs.

3. **Casts**
   Smart casts after checks; as? instead of a crash.

4. **Scope functions**
   let for null checks, apply and also return the object.

5. **Java interop**
   Platform types are unchecked: annotate with @Nullable.



Happy Learning :)